Stress Test Pipeline

Multi-Storm Stress Pipeline Model Documentation

MKM Research Labs

Version 1.0 — 18-March-2026

David K Kelly

Legal Notice

PROPRIETARY AND CONFIDENTIAL

This document, including all algorithms, models, methodology, formulas, calculations, and intellectual concepts contained herein, constitutes the exclusive intellectual property and confidential trade secrets of MKM Research Labs ("MKM").

Any usage, reproduction, distribution, modification, or disclosure of this document or any portion thereof, without the express prior written authorisation from MKM Research Labs is strictly prohibited and constitutes an infringement of intellectual property rights.

The document is provided on an "as is" basis. MKM Research Labs makes no representations or warranties of any kind, express or implied, regarding its accuracy, reliability, or suitability for any particular purpose.

Contents

1 Executive Summary

This document describes the Stress Test Pipeline (MKM-SP-001), an end-to-end orchestration system that generates spatially correlated multi-storm stress scenarios and trains per-gauge flood classifiers for use in portfolio stress testing and P&L estimation.

The pipeline ingests 10,000 synthetically generated storm sequences (from MKM-SS-001), parses the gauge portfolio with historical daily baselines, builds a spatial correlation model across all gauges, computes compound 168-hour gauge responses with seasonal base-level adjustment, populates per-gauge time series files (`gaugets/`), extracts the alert-breaching subset (`stress_storms.json`), and optionally trains a Gradient Boosting Machine (GBM) flood classifier per gauge.

The pipeline is invoked via `app.py port --stressm` and produces the following principal outputs:

- `storm_sequences.json` — 10,000 multi-storm sequences
- `sequences_summary.json` — lightweight metadata per sequence
- `sequence_gauge_summary.json` — per-sequence per-gauge peaks and flood flags (∼20–30 MB)
- `gaugets/*.json` — per-gauge base simulation + storm responses
- `stress_storms.json` — alert-breaching subset for UI consumption
- `data/output/stressm/*.joblib` — GBM classifiers (optional)
- `data/output/stressm/training_summary.json` — classifier metrics

Key design principle: The pipeline replaces several previously independent generation steps (storm generation, gauge time series, stress storm extraction, classifier training) with a single orchestrated flow that ensures consistency between the spatial correlation model, the compound gauge responses, and the downstream classifier training data.

2 Model Purpose and Scope

2.1 Purpose

The Stress Test Pipeline serves as the central orchestrator for multi-storm stress testing within the platform. It fulfils three functions:

- **Scenario generation:** Produce a large catalogue of spatially correlated, multi-storm gauge responses that capture compounding antecedent-wetness effects across the catchment.
- **Data population:** Generate the `gaugets/` directory and `stress_storms.json` file consumed by downstream modules (property time series, hazard curves, stress test UI, trading desk).
- **Classifier training:** Train per-gauge GBM flood classifiers on the full 10,000-sequence corpus, producing models with ∼1,680,000 training vectors per gauge.

2.2 Scope

- **In scope:** Storm sequence generation, spatial precipitation correlation, compound gauge response computation, seasonal baseline adjustment, gauge time series population, stress storm extraction, GBM classifier training.
- **Out of scope:** Property-level flood modelling, PRS pricing, depth-damage estimation, climate change projection.

2.3 Model Chain Position

The pipeline sits at the centre of the stress testing model chain:

$$\underbrace{\text{MKM-SI-001}}_{\text{Storm Intensity}} \rightarrow \underbrace{\text{MKM-SS-001}}_{\text{Sequences}} \rightarrow \boxed{\textbf{MKM-SP-001}\ \text{(this pipeline)}} \rightarrow \underbrace{\text{MKM-FC-001 v2.0}}_{\text{Classifiers}} \rightarrow \text{Stress P\&L}$$

Internally, MKM-SP-001 invokes MKM-SS-001 for sequence generation and delegates classifier training to MKM-FC-001 v2.0. The pipeline's primary contribution is the spatial correlation model and the compound response computation that connect these components.

3 Mathematical Framework

3.1 Pipeline Overview

The pipeline executes seven stages in sequence. The following diagram shows the data flow between stages and the files consumed and produced at each step.

```
+---------------------+
| Stage 1: Generate   |  --> storm_sequences.json
|   storm sequences   |  --> sequences_summary.json
+----------+----------+
           |
           v
+---------------------+
| Stage 2: Parse      |  <-- gauge.json
|   gauge portfolio   |  <-- gaugehd/*.json (baselines)
+----------+----------+
           |
           v
+---------------------+
| Stage 3: Build      |  (Haversine distance matrix,
|   spatial model     |   exponential correlation,
+----------+----------+   Cholesky decomposition)
           |
           v
+---------------------+
| Stage 4: Compound   |  --> sequence_gauge_summary.json
|   gauge responses   |
+----------+----------+
           |
           v
+---------------------+
| Stage 5: Populate   |  --> gaugets/*.json
```

```
|   gaugets           |  --> stress_storms.json
+----------+----------+
           |
           v (optional)
+---------------------+
| Stage 6: Train GBM  |  --> stressm/*.joblib
|   classifiers       |  --> training_summary.json
+---------------------+
```

3.2 Stage 1: Storm Sequence Generation

The pipeline delegates to `generate_event_set()` from the Storm Sequence Generator (MKM-SS-001) to produce N storm sequences (default $N = 10{,}000$). Each sequence contains 1–5 individual storms within a 168-hour event window, with calibrated durations, inter-storm gaps, and correlated intensity factors.

Output files:

- `storm_sequences.json` — full sequence catalogue (schema 2.0-multi-storm)
- `sequences_summary.json` — lightweight metadata per sequence

3.3 Stage 2: Gauge Portfolio Parsing

The gauge parser (`gauge_parser.py`) loads the gauge portfolio from `gauge.json` and enriches it with historical baselines from `gaugehd/`.

3.3.1 Gauge Extraction

The `_extract_gauges()` function handles both schema variants of `gauge.json`: a list under the `flood_gauges` key, or a dictionary keyed by gauge ID. Each record is normalised by `_parse_gauge()` to extract:

Field	Type	Source
`gauge_id`	string	`Header.GaugeID` or `gauge_id`
`lat`, `lon`	float	`Location.GaugeLatitude/Longitude`
`base_level`	float	gaugehd mean level, or 0.35 × flood_alert
`monthly_means`	dict	gaugehd monthly means (keys `"01"`…`"12"`)
`flood_alert`	float	`FloodStages.FloodAlert`
`flood_warning`	float	`FloodStages.FloodWarning`, or 1.33 × alert
`severe_warning`	float	`FloodStages.SevereFloodWarning`, or 1.58 × alert

3.3.2 Historical Baselines from gaugehd

The function `_load_gaugehd_baselines()` reads each `gauge_*_hd.json` file from the `gaugehd/` directory and extracts:

- `mean_level`: the annual mean daily water level (m), used as the default `base_level`.
- `monthly_means`: a dictionary mapping month strings (`"01"`...`"12"`) to the mean daily water level for that month. This enables seasonal base-level adjustment (Section **??**).

When gaugehd data is unavailable, the parser falls back to the heuristic base_level $= 0.35 \times$ flood_alert.

3.4 Stage 3: Spatial Correlation Model

The spatial model distributes catchment-average precipitation across gauges using an exponential correlation kernel calibrated to the Thames catchment.

3.4.1 Distance Matrix

An $N \times N$ Haversine distance matrix $\mathbf{D}$ is computed from the gauge coordinates:

$$D_{ij} = R \cdot 2 \arctan 2\big(\sqrt{a},\ \sqrt{1-a}\big)$$

where $R = 6{,}371\,\text{km}$ and

$$a = \sin^2\Big(\tfrac{\Delta\phi}{2}\Big) + \cos\phi_i \cos\phi_j \sin^2\big(\tfrac{\Delta\lambda}{2}\big)$$

3.4.2 Correlation Matrix

The exponential correlation matrix $\mathbf{C}$ is:

$$C_{ij} = \begin{cases} 1 & i = j \\ (1-\nu)\exp(-D_{ij}/r) & i \neq j \end{cases}$$

where $\nu = 0.05$ is the nugget (micro-scale variability) and r is the intensity-conditioned effective range:

$$r = r_0\,(1 + \rho \cdot \max(0,\ f - 1))$$

with $r_0 = 40\,\text{km}$ (base range), $\rho = 0.40$ (intensity scaling), and f the storm's intensity factor. Higher-intensity storms produce broader spatial coherence, increasing the effective correlation range.

3.4.3 Cholesky Decomposition

The Cholesky factor $\mathbf{L}$ satisfying $\mathbf{L}\mathbf{L}^\top = \mathbf{C}$ is computed and cached per distinct effective range (rounded to 0.1 km).

3.4.4 Spatially Correlated Multipliers

For each hour with non-zero precipitation, correlated per-gauge multipliers are sampled:

$$\mathbf{z} = \mathbf{L} \cdot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_N) \tag{1}$$

$$M_i = \exp\left(\sigma z_i - \tfrac{\sigma^2}{2}\right) \tag{2}$$

$$\tilde{M}_i = M_i \,/\, \overline{M} \tag{3}$$

where $\sigma = 0.40$ is the lognormal spatial coefficient of variation. Equation (**??**) produces mean-1 lognormal variates, and equation (**??**) normalises to preserve the catchment total: $\overline{\tilde{M}} = 1$ exactly.

Per-gauge precipitation at hour h:

$$P_i(h) = P_{\text{catch}}(h) \cdot \tilde{M}_i(h)$$

Parameter	Value	Description
`base_range_km`	40.0	Half of Thames corridor length
`nugget`	0.05	5% micro-scale variability
`rho_intensity`	0.40	Range scaling per unit intensity above 1
`sigma_lognormal`	0.40	40% spatial coefficient of variation

3.5 Stage 4: Compound Gauge Response

For each of the N sequences and each gauge, the pipeline computes a continuous 168-hour river level hydrograph that captures the compounding effect of multiple storms.

3.5.1 Precipitation Profile

Each storm within a sequence is assigned a triangular intensity profile. The peak position is at $p \cdot d$ hours from the storm start, where p is the `peak_position` parameter and d is the storm duration. The total precipitation per storm is preserved exactly.

3.5.2 Hydrological Routing

The precipitation series is routed through:

1. A soil moisture / groundwater hydrology model that tracks antecedent wetness between storms.
2. A quickflow-to-level first-order lag filter with asymmetric rise/fall:

$$\Delta_{\text{excess}}(h) = \begin{cases} \min(1,\ \alpha \cdot \beta) \cdot (Q(h) \cdot \gamma - e(h{-}1)) & Q(h) \cdot \gamma > e(h{-}1) \quad \text{(rising)} \\ \alpha \cdot (Q(h) \cdot \gamma - e(h{-}1)) & \text{otherwise} \quad \text{(falling)} \end{cases}$$

where:

- $e(h) = \max(0,\ e(h{-}1) + \Delta_{\text{excess}}(h))$ is the excess above base level (m)
- $Q(h)$ is the lagged quickflow (mm/h) at time $h - \tau_{\text{lag}}$
- $\gamma = 0.50$ is the quickflow-to-level scale (m per mm/h)
- $\alpha = 1/24$ is the hourly decay rate

- $\beta = 3.0$ is the rise factor (channels rise 3× faster than they recede)

The final river level at hour h is:

$$L_i(h) = b_i(m) + e_i(h)$$

where $b_i(m)$ is the seasonally adjusted base level for gauge i in month m (Section **??**).

3.5.3 Seasonal Base-Level Adjustment

When monthly means are available from gaugehd, the base level is adjusted to reflect seasonal variation:

$$b_i(m) = \begin{cases} \mu_i^{(m)} & \text{if monthly\_means available} \\ \bar{\mu}_i & \text{otherwise (annual mean)} \end{cases}$$

where $\mu_i^{(m)}$ is the mean daily water level for gauge i in month m, and $\bar{\mu}_i$ is the annual mean. This ensures that winter storms (higher baseline due to elevated groundwater) produce higher peak levels than equivalent summer storms.

The function `_seasonal_base_level(monthly_means, mean_level, month)` implements this lookup, falling back to the annual mean when monthly data is absent.

3.5.4 Threshold Exceedance

After computing peaks, each sequence is classified against the three flood thresholds:

$$\text{alert}_i = \mathbb{1}\left[\max_h L_i(h) \geq \text{flood\_alert}_i\right], \quad \text{warning}_i = \mathbb{1}\left[\max_h L_i(h) \geq \text{flood\_warning}_i\right], \quad \text{severe}_i = \mathbb{1}\left[\max_h L_i(h) \geq \right.$$

The output file `sequence_gauge_summary.json` (schema 2.1-spatial) records per-sequence:

- `peaks_m`: peak water level per gauge (m)
- `peak_hours`: hour of peak per gauge
- `alert`, `warning`, `severe`: boolean flags per gauge

3.6 Stage 5: Gaugets Population

This stage generates the `gaugets/` directory consumed by all downstream modules. It executes three sub-steps:

1. **Base simulation:** Generate 168-hour flood simulation files using `GaugeTimeSeriesGenerator`.
2. **Storm response merge:** Append per-gauge `storm_responses` from the sequence records into each `gaugets/GAUGE-*.json` file.
3. **Stress storm extraction:** Build `stress_storms.json` containing only the alert-breaching subset, via `generate_stress_storms()`.

3.7 Stage 6: GBM Classifier Training (Optional)

When `--train-classifier` is passed, the pipeline trains a GBM flood classifier per gauge. This stage delegates to `train_gauge_stressm_classifier()` which:

1. Re-computes the full 168-hour compound hydrograph per sequence (using the spatial model for gauge-specific precipitation).
2. Extracts four log-transformed features per hour: $\ell(t) = \ln(w/s)$, $\tau(t) = \ln((t+1)/168)$, $\Delta\ell(t)$, $\Delta^2\ell(t)$.
3. Labels each vector: $y = 1$ if $\max_{t' \geq t} w(t') \geq s$.
4. Delegates to `FloodClassifierTrainer.train_gauge()` from MKM-FC-001 for the actual GBM fitting and evaluation.

If no sequences breach the severe threshold for a given gauge, the labelling falls back to the flood alert level. This ensures the classifier is always trained with both positive and negative examples.

Total training corpus per gauge: $10{,}000 \times 168 = 1{,}680{,}000$ vectors.

4 Input Parameters and Calibration

4.1 Pipeline Parameters

Parameter	Default	Description
`count`	10,000	Number of storm sequences to generate
`seed`	42	Random seed for reproducibility
`catchment_id`	`thames`	Catchment identifier
`gauge_id`	`None`	Single-gauge mode (optional)
`train_classifier`	`False`	Enable GBM training
`verbose`	`False`	Extra progress logging

4.2 Hydraulic Routing Parameters

Parameter	Default	Description
`sensitivity`	1.0	Gauge response multiplier
`quickflow_to_level_scale`	0.50	mm/h quickflow to m level rise
`response_lag_hours`	2	Hours before quickflow reaches gauge
`response_decay_hours`	24.0	Hydraulic time constant (hours)
`rise_factor`	3.0	Asymmetric rise/fall ratio

4.3 Base-Level Heuristics

When gaugehd data is absent, the following heuristic ratios are applied:

Threshold	Ratio	Description
`base_level`	$0.35 \times$ alert	Normal water level
`flood_warning`	$1.33 \times$ alert	Warning threshold
`severe_warning`	$1.58 \times$ alert	Severe threshold

5 Implementation Details

5.1 Source Code

Module	Responsibility
`src/port/src/stressm/pipeline.py`	Main orchestrator: `generate_stressm()`, gaugets population, summary writing
`src/port/src/stressm/gauge_parser.py`	Gauge extraction, gaugehd baseline loading, seasonal base-level adjustment
`src/port/src/stressm/classifier.py`	Per-gauge GBM training: hydrograph synthesis, feature extraction, delegation to `FloodClassifierTrainer`
`src/port/src/stressm/reporting.py`	Single-gauge progression report (threshold exceedance, percentiles, category breakdowns)
`src/port/src/stressm/__init__.py`	Package exports

5.2 External Dependencies

The pipeline delegates to several external modules:

- `port.src.storm_multi` — Storm sequence generation, `SpatialCorrelationModel`, `SequenceGaugeParams`, `compute_sequence_gauge_response()`
- `port.src.gauge.gaugets` — `GaugeTimeSeriesGenerator` for base 168h simulation
- `port.src.gauge.stress_storms` — `generate_stress_storms()` for alert-breaching extraction
- `models.stress.flood_classifier` — `FloodClassifierTrainer` for GBM fitting and serialisation

5.3 Single-Gauge Mode

When `gauge_id` is specified, the pipeline restricts the response computation to a single gauge while still generating all sequences and building the full spatial model (which requires all gauge locations for the correlation matrix). A detailed progression report is printed showing threshold exceedance rates, peak level percentiles, and breakdowns by intensity category and sequence type.

5.4 Output Schema

The principal output file `sequence_gauge_summary.json` uses schema version `2.1-spatial`:

Field	Description
schema_version	"2.1-spatial"
catchment_id	Catchment identifier
num_sequences	Total number of sequences
num_gauges	Number of gauges processed
gauge_ids	Ordered list of gauge IDs
sequences[]	Array of per-sequence records
sequences[].peaks_m	Peak level per gauge (m), array of floats
sequences[].peak_hours	Hour of peak per gauge, array of ints
sequences[].alert	Alert exceedance per gauge, array of bools
sequences[].warning	Warning exceedance per gauge, array of bools
sequences[].severe	Severe exceedance per gauge, array of bools

6 Data Flow

6.1 Input Files

File	Description	Consumed By
gauge.json	Gauge portfolio (IDs, locations, thresholds)	Stage 2
gaugehd/*.json	Historical daily water levels per gauge	Stage 2

6.2 Output Files

File	Description	Produced By
storm_sequences.json	Full sequence catalogue	Stage 1
sequences_summary.json	Sequence metadata	Stage 1
sequence_gauge_summary.json	Per-sequence per-gauge peaks	Stage 4
gaugets/*.json	Per-gauge time series + responses	Stage 5
stress_storms.json	Alert-breaching subset	Stage 5
stressm/*.joblib	GBM classifiers	Stage 6
training_summary.json	Classifier metrics	Stage 6

6.3 Downstream Consumers

- **Property time series** (propertyts/): reads gaugets/ for gauge-level storm responses
- **Hazard curve builder**: reads gaugets/ for exceedance statistics
- **Trading desk stress tab**: reads stress_storms.json for storm dropdown; reads stressm/*.joblib for flood probability
- **Animation / simulation**: reads gaugets/ for 168h hydrograph

7 Sensitivity Analysis

The pipeline's output is sensitive to the following parameters:

1. **Spatial correlation range** (`base_range_km`): Larger ranges produce more uniform precipitation across gauges, reducing spatial diversity in stress scenarios.
2. **Lognormal sigma** (`sigma_lognormal`): Higher values increase gauge-to-gauge precipitation variability. At $\sigma = 0$, all gauges receive identical precipitation.
3. **Seasonal baselines**: Winter months (November–February) produce higher base levels from gaugehd data, systematically increasing peak levels and alert exceedance rates for winter-starting sequences.
4. **Rise factor** ($\beta = 3.0$): Controls the asymmetry of hydrograph shape. Higher values produce sharper peaks.
5. **Sequence count** (N): Affects the statistical stability of exceedance rates and classifier training quality.

8 Model Limitations and Known Weaknesses

1. **Sequential processing:** Sequences are processed one at a time. For 10,000 sequences $\times$ 52 gauges, the compound response computation takes approximately 10–15 minutes on a single core. No parallelisation is currently implemented.
2. **Stationary correlation:** The spatial correlation kernel uses a single exponential form with fixed parameters. Real precipitation fields exhibit anisotropy and non-stationary correlation structures.
3. **Synthetic precipitation:** The triangular intensity profile is a simplification. Real storms exhibit irregular, multi-modal precipitation patterns.
4. **No feedback:** River levels do not feed back into the precipitation or hydrology model. In reality, floodplain storage attenuates downstream peaks.
5. **Heuristic thresholds:** When gaugehd data is absent, the $0.35 \times$ alert, $1.33 \times$ alert, and $1.58 \times$ alert ratios are uncalibrated assumptions.
6. **Classifier re-computation:** The classifier training stage re-computes all compound hydrographs from scratch rather than reusing Stage 4 results, approximately doubling the computation time when `--train-classifier` is enabled.
7. **Single-catchment calibration:** All default parameters (spatial correlation, hydraulic routing, heuristic thresholds) are calibrated for the Thames catchment and may not generalise.

9 Change History

Version	Date	Description
1.0	18-Mar-2026	Initial pipeline documentation: 7-stage architecture, spatial correlation model, compound gauge response, seasonal baselines, gaugets population, GBM classifier integration